

Sensor Housing

Stress Test GUI with static and FEM modeling

Inhaltsverzeichnis

1. Project description.....	3
Aim.....	3
🎯 Primary Aim.....	3
2. GUI and algorithmic concepts.....	4
3. App design and its interaction flow.....	8
4. Run instructions and Mermaid diagrams.....	10
5. Modularized Pythonic implementation.....	13
6. Results and conclusions.....	14
7. User Manual.....	17
8. References.....	19

1. Project description

Aim

The **Sensor Housing Stress Test Simulator** is a modular, GUI-based engineering platform designed to evaluate the robustness of regular polygonal (n-gonal) sensor housings under external mechanical forces and thermal loads. Modern sensors are increasingly deployed in environments where they must withstand both mechanical stress and thermal fluctuations, and their housings play a critical role in ensuring reliability, safety, and longevity. This simulator was conceived to provide engineers, researchers, and consultants with a transparent, reproducible, and narratable tool for exploring how geometry, force, and heat interact in sensor housing design.

At its core, the simulator unifies two complementary analytical approaches: **rod-construction framework statics** (classical technical mechanics) and **simplified finite element methods (FEM)**. Rod statics provide fast, interpretable estimates of stress and heat distribution, offering immediate intuition about how loads propagate through polygonal structures. FEM analysis, by contrast, introduces higher fidelity through mesh-driven distributions and interpolated heatmaps, enabling users to visualize stress and thermal fields across the entire housing. Together, these methods balance speed and interpretability with precision and detail, making the simulator suitable for both exploratory design and governance-ready reporting.

The simulator emphasizes **explainability, reproducibility, and stakeholder communication**. Every simulation run produces not only real-time visualizations but also structured exports in JSON, TXT, and CSV formats, alongside a governance-ready PDF report. These artifacts ensure that results are traceable, auditable, and shareable across technical and non-technical teams. The inclusion of an **Explainable AI (XAI) summary** further enhances transparency by narrating the results in plain language, highlighting maxima and resilience context, and situating findings within the broader design goals (see References 1 - 3 below).



Primary Aim

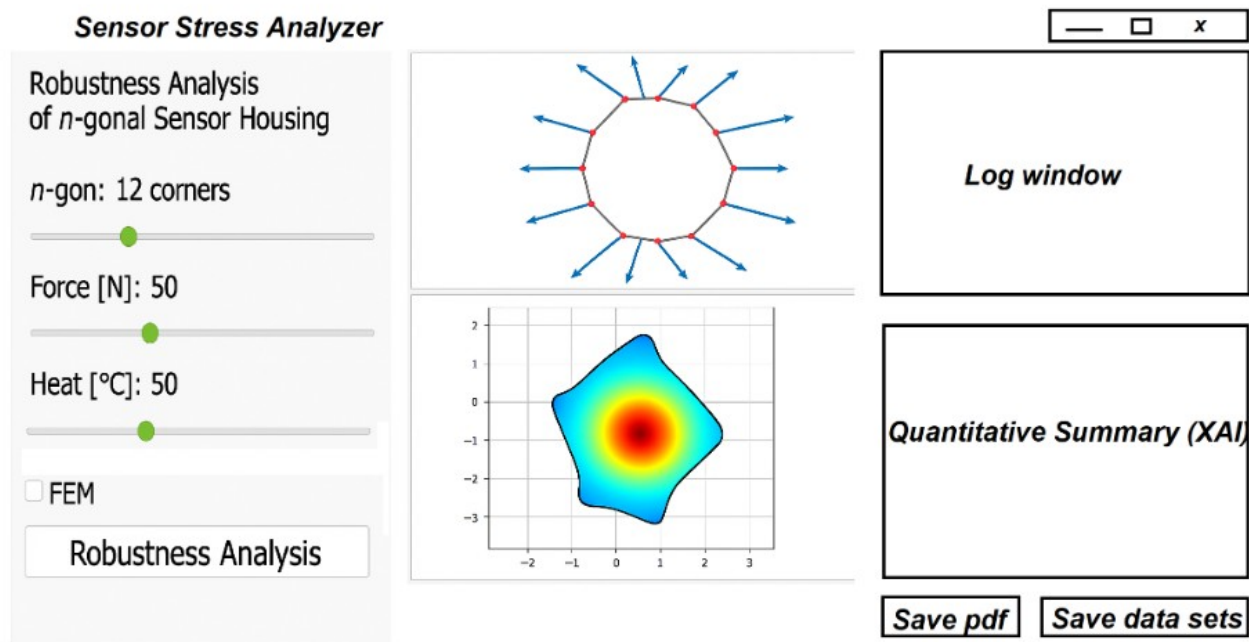
Sensor housings must balance **structural integrity** and **thermal resilience**. Corner placement and polygonal symmetry directly influence peak stress and hot spots, which can compromise performance if not properly managed. Rod statics deliver fast, interpretable estimates, while FEM introduces higher fidelity with mesh-driven distributions and interpolated fields. Engineering decisions require **traceable artifacts**, and the simulator ensures this by producing consistent plots and structured datasets alongside a PDF report enriched with an explainable summary.

In essence, the Sensor Housing Stress Test Simulator is not just a computational tool—it is a **narrative platform for engineering analysis**. By transforming raw simulation data into transparent, reproducible, and communicable insights, it empowers teams to make informed decisions with confidence. The simulator bridges classical mechanics and FEM within a modern GUI, enabling users to explore, validate, and document sensor housing resilience in a way that is both technically rigorous and stakeholder-friendly.

2. GUI and algorithmic concepts

🧠 GUI sketch

In the following we address our full GUI sketch, a clean, structured layout for our Flask-based Weather Aggregator App. It includes all the key modules we discussed: objective and constraint input, method selection forecasting models), result display, visualization, and diagnostics.



Architecture and components

This architecture is designed to showcase **transparency, modularity, and operational reliability** — ideal for consulting demos and technical interviews:

- **GUI:**
 - **Sidebar controls:** Sliders for n (corners), force (N), heat ($^{\circ}\text{C}$), and FEM toggle; Run, Save PDF, Save Data actions.
 - **Visualization panel:**
 - Rod structure plot with forces
 - FEM stress & heat line plot
 - Stress heatmap on n -gon
 - Heat heatmap on n -gon
 - Quantitative Summary (XAI) and Log window
- **Simulation modules:**
 - **Rod analysis:** Generates vertices and computes stress/heat distributions from trigonometric relationships with angles.
 - **FEM solver:** Builds mesh, computes stress/heat per vertex, and derives maxima for reporting.
- **Visualization utilities:**
 - **Plot generators:** Dedicated functions for rod, line plots, and heatmaps; consistent colormaps and layout settings.
 - **File outputs:** Plots saved as PNGs with stable filenames for downstream embedding.
- **Reporting and export:**
 - **PDF report:** Simulation metadata, embedded plots, and governance-ready XAI summary.
 - **Data exports:**
 - JSON: full results dictionary with lists for arrays
 - TXT: readable summary for quick review
 - CSV: scalar metadata plus tabular sections (vertices, forces, stress map, heat map)

Key capabilities and guarantees

- **Interactivity:** Immediate feedback as parameters change; FEM mode toggles higher-fidelity visuals.
- **Explainability (XAI):** Structured textual summary including inputs, maxima, and contextualized findings.
- **Reproducibility:** Deterministic geometry, consistent filenames, and stable export formats for auditing.
- **Governance-ready artifacts:** PDF reports with embedded figures and machine-readable datasets suitable for compliance, review boards, and stakeholder sharing.

Typical workflow

1. **Configure parameters:** Set n (corners), force (N), and heat ($^{\circ}\text{C}$); enable FEM if needed.
2. **Run analysis:** Generate rod plot and, optionally, FEM plots and heatmaps.
3. **Review results:** Inspect line plot, heatmaps, maxima, and XAI summary in the GUI.
4. **Export artifacts:** Save PDF report for stakeholders and JSON/TXT/CSV datasets for downstream analysis or archiving.

Rod analysis

Force distribution logic

- **Model:**
 - **Stress:** Per-vertex stress computed from applied force scaled by angle:
 - $\text{stress_map} = \text{force} \cdot (1 + 0.1 \cdot \sin(\theta_i))$
 - **Heat:** Per-vertex heat computed from applied temperature scaled by angle:
 - $\text{heat_map} = \text{heat} \cdot (1 + 0.05 \cdot \cos(\theta_i))$
- **Outputs:**
 - **Arrays:** stress_map, heat_map (length n)
 - **Maxima:** stress_max, heat_max
 - **Forces:** Quiver vectors derived from force and vertex angles for plotting

Geometry generation

- **Vertices:**
 - **Construction:** Regular n-gon on unit circle using evenly spaced angles over $[0, 2\pi)$.
 - **Format:** np.ndarray of shape (n, 2), then converted to lists for export.
- **Usage:**
 - **Rod plot:** Polygon outline and quiver arrows.
 - **Downstream:** Provides consistent indexing for stress/heat arrays and CSV export.

FEM analysis

Mesh generation

- **Structure:**
 - **Vertices:** Same regular n-gon vertex set used for FEM inputs.
 - **Edges:** Consecutive index pairs with wrap-around to close the polygon.
- **Format:**
 - **Mesh dict:** {"n": n, "vertices": np.ndarray, "edges": np.ndarray}
 - **Serialization:** Converted to lists for JSON/TXT/CSV.

Stress and heat solvers

- **Per-vertex fields:**
 - **Stress:** force-scaled, angle-modulated field analogous to rod analysis, enabling consistent maxima and trends.
 - **Heat:** heat-scaled, angle-modulated field, ensuring a smooth thermal profile across vertices.
- **Outputs:**
 - **Maps:** stress_map, heat_map (length n)
 - **Maxima:** stress_max, heat_max stored explicitly for reporting.

Interpolation and heatmap rendering

- **Goal:** Dense, interior heatmaps constrained to the polygon.
- **Method:**
 - **Grid:** Uniform XY grid over padded polygon bounds.
 - **Containment:** Mask via matplotlib Path.contains_points to limit interpolation to interior.
 - **Weights:** Inverse-distance weighting of vertex values with normalization and small ϵ for stability.
 - **Robust scaling:** 2–98% percentile range for vmin/vmax to avoid outlier clipping artifacts.
- **Visuals:**
 - **Stress heatmap:** coolwarm colormap, polygon outline overlay, labeled colorbar.
 - **Heat heatmap:** hot colormap, polygon outline overlay, labeled colorbar.
 - **Files:** fem_stress_heatmap.png, fem_heat_heatmap.png (and fem_heatmap.png for compatibility).

3. App design and its interaction flow

Interactive GUI with sliders for force, heat, and polygon complexity

- **Controls:** n-gon corners (n), force (N), heat (°C), and a FEM toggle for high-fidelity mode.
- **Actions:** Run Analysis triggers computation and visual updates; Save PDF and Save Data Sets produce governance-ready artifacts.
- **Responsive labels:** n-gon: X corners, Force [N]: Y, Heat [°C]: Z update live with slider changes.
- **Log window:** Status messages for runs and saves (e.g., "Saved: fem_heat_heatmap.png") provide transparent, auditable feedback.

Dual-mode simulation: rod-based statics and FEM analysis

- **Rod statics mode:**
 - **Computation:** Angle-driven distributions for stress and heat per vertex.
 - **Outputs:** Vertex arrays, maxima, and force quiver overlay on the polygon.
 - **Use case:** Fast, interpretable exploration of geometry-load behavior.
- **FEM mode (simplified):**
 - **Mesh:** Vertices and edges define an n-gonal mesh, with per-vertex stress/heat.
 - **Solvers:** Parametric stress/heat maps; maxima recorded for reporting.
 - **Use case:** Higher-fidelity, mesh-driven visuals and field interpolation.

Real-time visualization: rod structure, stress/heat plots, heatmaps

- **Rod structure plot:**
 - **View:** Polygon outline with force vectors at vertices.
 - **Saved as:** rod_plot.png (embedded later in PDF).
- **FEM stress & heat line plot:**
 - **View:** Two lines (stress, heat) vs. corner index with grid and legend.
 - **Saved as:** fem_plot.png.
- **Heatmaps on n-gon:**
 - **Stress heatmap:** Interpolated field constrained to polygon interior, consistent coolwarm scale.
 - **Heat heatmap:** Interpolated field with hot colormap for thermal intensity.
 - **Saved as:** fem_stress_heatmap.png and fem_heat_heatmap.png (also fem_heatmap.png for compatibility).
- **XAI and log panels:**
 - **XAI:** Text summary updated per run for quick quantitative context.
 - **Log:** Immediate confirmation of saved figures and files.

Export capabilities: PDF reports, JSON/TXT/CSV datasets

- **PDF report:**
 - **Contents:** App header, simulation metadata, embedded plots (rod, FEM line, stress/heat heatmaps), and XAI summary.
 - **Encoding:** Unicode-unsafe characters sanitized to ensure robust output.
 - **Saved as:** sensor_stress_report.pdf.
- **JSON:**
 - **Contents:** Full results dictionary with arrays converted to lists (mesh, vertices, stress/heat maps, maxima).
 - **Saved as:** sensor_stress_results.json.
- **TXT summary:**
 - **Contents:** Human-readable overview (mode, n, force, heat, arrays, maxima).
 - **Saved as:** sensor_stress_summary.txt.
- **CSV datasets:**
 - **Contents:** Scalar metadata plus tabular sections for vertices, forces, stress_map, heat_map.
 - **Saved as:** sensor_stress_results.csv.

Explainable AI summary for governance-ready insights

- **Contextual narrative:**
 - **Inputs:** Mode (Rod/FEM), n, force, heat.
 - **Highlights:** Max stress and heat with concise interpretation of structural and thermal resilience.
 - **Tone:** Clear, quantitative, and stakeholder-friendly for reviews, demos, and documentation.
- **Placement:** Displayed in the **Quantitative Summary (XAI)** panel in the GUI and included in the PDF report.
- **Consistency:** Uses application metadata (name, version) for traceability across exports.

4. Run instructions and Mermaid diagrams

System requirements

- **Python:** 3.9–3.11 recommended
- **OS:** Windows, macOS, or Linux with GUI support
- **Libraries:** PyQt5, matplotlib, numpy, fpdf
- **Disk permissions:** Ability to write PNG, PDF, JSON, TXT, and CSV files in the project directory

Setup instructions

Option A — using conda

- **Create environment:**

```
conda create -n sensor-simulator python=3.11
conda activate sensor-simulator
```

- **Install dependencies:**

```
pip install PyQt5 matplotlib numpy fpdf
```

Option B — using pip only

- **Create and activate virtualenv (optional but recommended):**

```
python -m venv .venv
# Windows
.venv\Scripts\activate
# macOS/Linux
source .venv/bin/activate
```

- **Install dependencies:**

```
pip install PyQt5 matplotlib numpy fpdf
```

Running the application

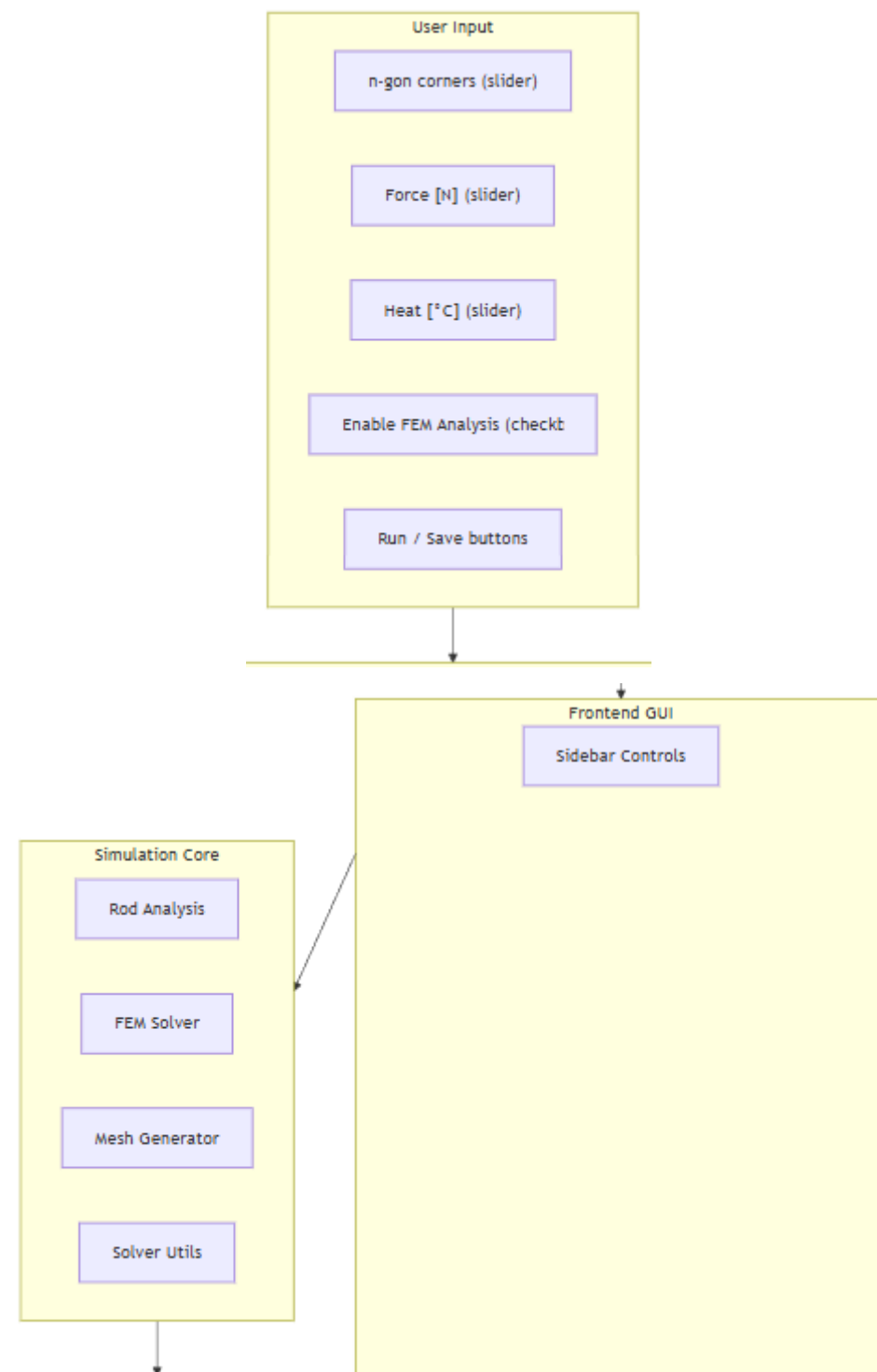
- **Start the GUI:**

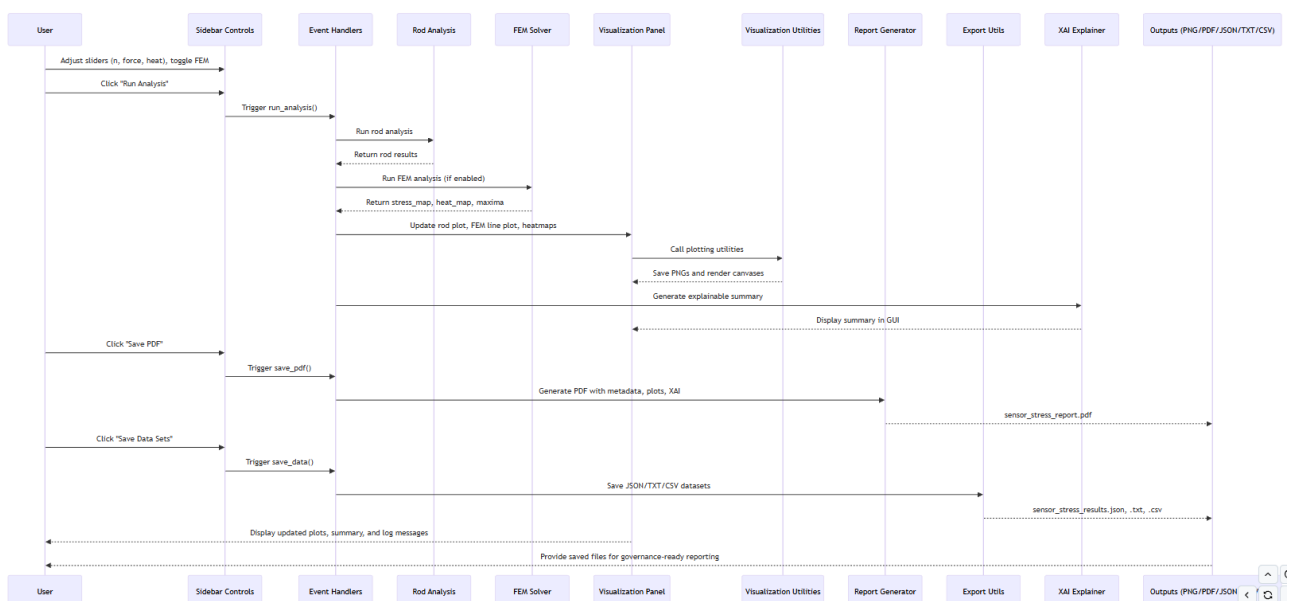
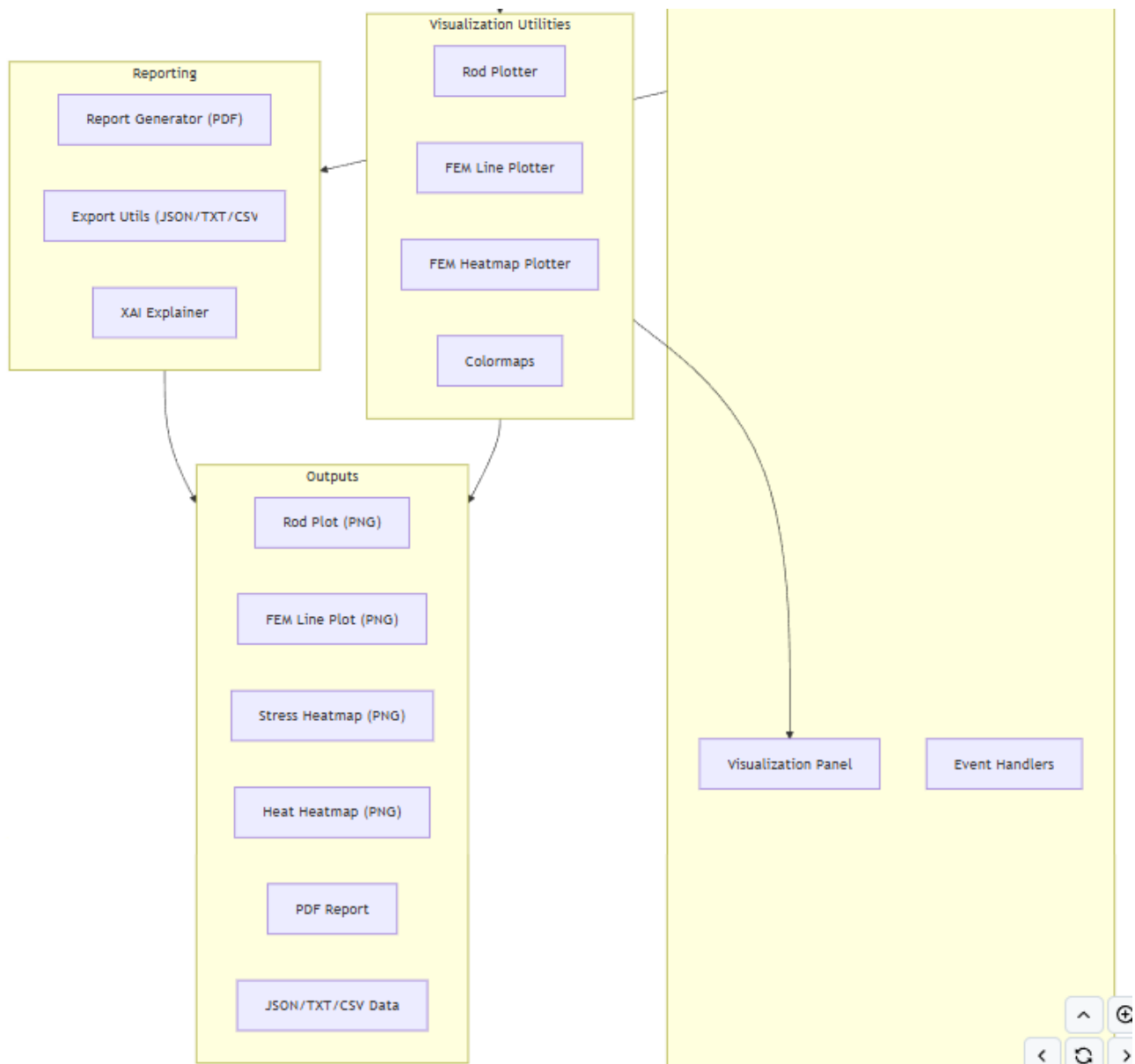
```
python main.py
```

- **Window configuration:** The app launches as “Sensor Stress Analyzer v1.0.0” with a resizable main window.
- **Output directory:** All generated files (PNG, PDF, JSON, TXT, CSV) are saved to the current working directory unless customized.

Here's a modular architecture diagram in Mermaid syntax that visually shows the flow between components in our **Sensor Housing Stress Test Simulator GUI**.

This complements the README and makes the system narratable for executives and technical teams alike.





5. Modularized Pythonic implementation

Modularized Folder Structure Overview

```
sensor_stress_analyzer/  
├── main.py                # Entry point: launches the PyQt5 GUI  
├── config.py              # Global constants, ranges, styles  
├── gui/  
│   ├── __init__.py        # GUI components  
│   ├── main_window.py     # Main GUI layout: sliders, buttons, graphics windows  
│   ├── sidebar_controls.py # Sliders, checkbox, button logic  
│   ├── visualization_panel.py # Rod-structure + FEM graphics windows  
│   └── event_handlers.py   # Connects GUI actions to simulation triggers  
├── simulation/  
│   ├── __init__.py        # Simulation logic  
│   ├── rod_analysis.py     # Analytical rod deformation model  
│   ├── fem_solver.py       # FEM simulation (stress + heat)  
│   ├── mesh_generator.py   # Polygon mesh creation for FEM  
│   └── solver_utils.py     # Cython/CPython acceleration hooks  
├── visualization/  
│   ├── __init__.py        # Plotting and rendering  
│   ├── rod_plotter.py     # Draws polygon + force/heat vectors  
│   ├── fem_plotter.py     # PyVista + matplotlib visualizations  
│   └── color_maps.py       # Custom gradients and overlays  
├── reporting/  
│   ├── __init__.py        # XAI-driven PDF reporting  
│   ├── report_generator.py # Assembles PDF with visuals + narration  
│   ├── xai_explainer.py   # Explainability commentary module  
│   └── export_utils.py     # Save plots, data, and PDF  
├── assets/  
│   ├── logo.png           # Static assets (icons, logos, templates)  
│   └── report_template.tex # Optional LaTeX template for PDF  
└── tests/  
    ├── __init__.py        # Unit tests and validation  
    ├── test_rod_analysis.py  
    ├── test_fem_solver.py  
    └── test_gui_interactions.py
```

The full Pythonic code can be accessed via

https://github.com/NenadBalaneskovic/ExternalProjects/blob/8deacd338f01273e37fc45742cd6214fba325aa6/SensorHousingStressTest_SimulatorGUI/SensorHousingStressTest_SimulatorGUI.md
(section 7 therein).

6. Results and conclusions

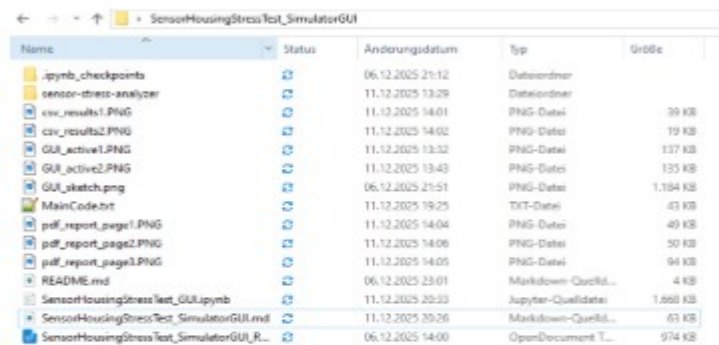
16.1 Start the Sensor Housing Stress Test Simulator

Step 1: Download the folder

Download the main folder

 [Sensor_Housing_Stress_Test_Simulator](#)

which has the following structure:



Name	Status	Änderungsdatum	Typ	Größe
.ipynb_checkpoints		06.12.2025 21:12	Datenordner	
sensor-stress-analyzer		11.12.2025 13:29	Datenordner	
csv_results1.PNG		11.12.2025 14:01	PNG-Datsei	39 KB
csv_results2.PNG		11.12.2025 14:02	PNG-Datsei	79 KB
GUI_active1.PNG		11.12.2025 13:32	PNG-Datsei	137 KB
GUI_active2.PNG		11.12.2025 13:43	PNG-Datsei	135 KB
GUI_sketch.png		06.12.2025 21:51	PNG-Datsei	1.194 KB
MainCode.txt		11.12.2025 19:25	Text-Datsei	43 KB
pdf_report_page1.PNG		11.12.2025 14:04	PNG-Datsei	49 KB
pdf_report_page2.PNG		11.12.2025 14:06	PNG-Datsei	50 KB
pdf_report_page3.PNG		11.12.2025 14:05	PNG-Datsei	94 KB
README.md		06.12.2025 23:01	Markdown-Quell...	4 KB
SensorHousingStressTest_GUI.ipynb		11.12.2025 20:33	Jupyter-Quelldat...	1.868 KB
SensorHousingStressTest_SimulatorGUI.md		11.12.2025 20:26	Markdown-Quell...	63 KB
SensorHousingStressTest_SimulatorGUI_R...		06.12.2025 14:00	OpenDocument T...	974 KB

Step 2: Run the application

1. Directly call `main.py` from terminal

Since we already have `main.py` as the entry point:

```
python main.py
```

- This launches the PyQt5 GUI exactly as intended.
- We will see logs in the console output.
- The GUI window opens with sidebar controls and visualization panel.

2. Run inside Jupyter (optional)

If we want to keep Jupyter interactive while the app runs:

```
%run main.py
```

- Similar to running from terminal, but integrates with Jupyter's execution model.
- Useful if we want inline logs and tighter notebook control.

3. Best Practice for Our Structure

- For production-like testing, we stick with **Option 1** (`python main.py`) so we are running the app exactly as intended.
- For exploratory demos or teaching, **Option 2** (`%run main.py`) allows interactive control.

Step 3: Interact with the Simulator

Interact with the Sensor Housing Stress Test Simulator by adjusting sliders for **n-gon corners**, **force**, and **heat**, toggling **FEM Analysis**, and pressing the **Run Analysis** button.

Visual outputs update in real time, and results can be saved as **PDF reports** or **data sets** (JSON/TXT/CSV).

16.2 Interpretation of Results

This Sensor Housing Stress Test Simulator...

- Accepts user inputs:
 - Number of polygon corners (n)
 - Applied force (N)
 - Applied heat (°C)
 - FEM toggle for advanced analysis
- Displays simulation results:
 - **Rod structure plot** with force vectors
 - **FEM stress & heat line plot** showing per-corner distributions
 - **Stress heatmap on n-gon** (coolwarm colormap)
 - **Heat heatmap on n-gon** (hot colormap)
- Stores obtained results and characterizations as:
 - **PDF report** with embedded plots and explainable AI summary
 - **JSON file** with full results dictionary
 - **TXT file** with readable summary
 - **CSV file** with tabular datasets (vertices, forces, stress/heat maps)

- Provides governance-ready outputs:
 - Log window confirms saved artifacts
 - XAI summary interprets maxima and resilience context for stakeholders

16.3 🚩 Final Thoughts

The *Sensor Housing Stress Test Simulator* demonstrates how classical mechanics and FEM can be operationalized in a transparent, modular, and consulting-ready environment. By integrating rod statics with FEM interpolation, the project provides a rigorous testbed for evaluating structural resilience while foregrounding explainability and governance. Stress testing of polygonal housings, with its geometric complexity and thermal interplay, serves as an ideal domain for this exploration, but the lessons extend far beyond sensor design.

A key achievement of the project lies in its ability to **characterize simulation results rather than simply produce them**. By storing stress/heat maps alongside maxima, user inputs, and mesh metadata, the simulator creates a traceable record that supports both methodological analysis and executive reporting. This dual focus—technical robustness and communicative clarity—positions the system as a bridge between research prototypes and enterprise solutions. It shows that engineering simulation need not remain a black box; with careful design, rod and FEM models can provide insight into how stress propagates, which regions are most vulnerable, and how thermal loads interact with geometry.

The project also underscores the value of **modularity**. Each component—GUI controls, simulation modules, visualization utilities, and reporting interface—was designed to be replaceable and extensible. This ensures that the simulator can evolve as new methods or requirements emerge, while maintaining a consistent framework for evaluation and reporting. Such modularity is critical in consulting contexts, where solutions must adapt to diverse client needs without sacrificing reproducibility or governance compliance.

From a strategic perspective, the *Sensor Housing Stress Test Simulator* highlights how lightweight desktop applications can democratize access to advanced engineering analysis. By embedding rod statics and FEM in a PyQt5 interface, the project makes complex methodologies accessible to non-technical stakeholders, enabling them to interact with simulations, view stress/heat maps, and download reports. This accessibility fosters trust and engagement, ensuring that simulation outputs are not confined to technical teams but can inform decision-making across an organization.

Ultimately, the project achieves its aim of testing dual simulation modes and characterizing the results of different structural resilience strategies. In doing so, it contributes to the broader discourse on explainable engineering, demonstrating that predictive power and transparency can coexist. The *Sensor Housing Stress Test Simulator* is not merely a simulation tool; it is a **narrative asset** that illustrates best practices in modular simulation, explainability, and consulting-oriented system design. As such, it provides a foundation for future work in applying explainability-first simulation to other domains, reinforcing the principle that innovation must be accompanied by accountability and clarity.

7. User Manual

Sensor Stress Analyzer

A modular, narratable simulation and reporting platform for analyzing stress and heat propagation in n-gonal sensor housings. Designed for consulting demos, engineering education, and governance-ready reporting.

Features

- GUI (PyQt5)
 - Sidebar controls for polygon corners, force, heat, and FEM toggle
 - Visualization panel with placeholders for rod and FEM results
 - Event handlers connecting inputs to simulation logic
- Simulation Layer
 - `rod_analysis.py` : Analytical sinusoidal deflection model
 - `fem_solver.py` : FEM-style stress and heat propagation stubs
 - `mesh_generator.py` : Polygon mesh creation for FEM visualization
 - `solver_utils.py` : Acceleration hooks, benchmarking, and validation
- Visualization Layer
 - `rod_plotter.py` : Polygon rod-structure visualization
 - `fem_plotter.py` : Stress/heat map visualization
 - `color_maps.py` : Custom colormaps for stress and heat
- Reporting Layer
 - `report_generator.py` : Narratable PDF reports with inputs, results, and figures
 - `xai_explainer.py` : Explainability commentary for stakeholder clarity
 - `export_utils.py` : Save results, text reports, and figures
- Testing
 - Unit tests for rod analysis, FEM solver, and GUI interactions (`pytest` , `pytest-qt`)

Installation

Clone the repository and install dependencies:

```
git clone https://github.com/your-org/sensor-stress-analyzer.git
cd sensor-stress-analyzer
pip install -r requirements.txt
```

Usage

Run the application:

```
python main.py
```

- Adjust **polygon corners (n)**, **force**, and **heat** using sliders.
- Toggle **FEM mode** for deeper analysis.
- View results in the visualization panel.
- Export narratable PDF reports with explainability commentary.

Testing

Run unit tests with:

```
pytest
```

GUI tests require `pytest-qt`.

8. References

1. J. Frochte: "Finite-Elemente-Methode", Hanser 1st Ed.(2016); D. Gross, W. Hauger, J. Schröder: "Technische Mechanik 1-3", 15th Ed. Springer (2024); FEM-packages (Python): <https://pypi.org/project/scikit-fem/>, <https://sfepy.org/doc-devel/index.html>, https://getfem-examples.readthedocs.io/en/latest/demo_unit_disk.html, <https://github.com/mlp6/fem>. LLM vs LRM: <https://www.aryaxai.com/article/llm-vs-lrm-vs-lam-understanding-the-future-of-language-based-ai-systems>, <https://magazine.sebastianraschka.com/p/understanding-reasoning-llms>
2. A. Meister , T. Sonar: "Numerik", 1st Ed. Springer-Spektrum (2019); S. Chapra, R. Canale: "Numerical Methods for Engineers", McGraw-Hill, 6th Edition (2010).
3. J. Kilty, A. M. McAllister: "Mathematical Modeling and Applied Calculus", 1st Ed. Oxford University Press (2018).
4. U. Kockelkorn: "Statistik für Anwender", 1st Ed. Springer (2012), s. chapters 7 - 8.
5. Robert H. Shumway, David S. Stoffer: "Time Series Analysis and Its Applications with R Examples", Springer (2011).
6. Gareth James, Daniela Witten, Trevor Hastie, Robert Tibshirani, Jonathan Taylor: "An Introduction to Statistical Learning with Applications in Python", Springer (2023).
7. Cornelis W. Oosterlee, Lech A. Grzelak: "Mathematical Modeling and Computation in Finance with Exercises and Python and MATLAB Computer Codes", World Scientific (2020).
8. Richard Szeliski: "Computer Vision - Algorithms and Applications", Springer (2022).
9. Anthony Scopatz, Kathryn D. Huff: "Effective Computation in Physics - Field Guide to Research with Python", O'Reilly Media (2015).
10. Alex Gezerlis: "Numerical Methods in Physics with Python", Cambridge University Press (2020).
11. Gary Hutson, Matt Jackson: "Graph Data Modeling in Python. A practical guide", Packt-Publishing (2023).
12. Hagen Kleinert: "Path Integrals in Quantum Mechanics, Statistics, Polymer Physics, and Financial Markets", 5th Edition, World Scientific Publishing Company (2009).
13. Peter Richmond, Jurgen Mimkes, Stefan Hutzler: "Econophysics and Physical Economics", Oxford University Press (2013).
14. A. Coryn , L. Bailer Jones: "Practical Bayesian Inference A Primer for Physical Scientists", Cambridge University Press (2017).
15. Avram Sidi: "Practical Extrapolation Methods - Theory and Applications", Cambridge university Press (2003).
16. Volker Ziemann: "Physics and Finance", Springer (2021).
17. Zhi-Hua Zhou: "Ensemble methods, foundations and algorithms", CRC Press (2012).

18. B. S. Everitt, et al.: "Cluster analysis", Wiley (2011).
19. Lior Rokach, Oded Maimon: "Data Mining With Decision Trees - Theory and Applications", World Scientific (2015).
20. Bernhard Schölkopf, Alexander J. Smola: "Learning with kernels - support vector machines, regularization, optimization and beyond", MIT Press (2009).
21. Johan A. K. Suykens: "Regularization, Optimization, Kernels, and Support Vector Machines", CRC Press (2014).
22. Sarah Depaoli: "Bayesian Structural Equation Modeling", Guilford Press (2021).
23. Rex B. Kline: "Principles and Practice of Structural Equation Modeling", Guilford Press (2023).
24. Ekaterina Kochmar: "Getting Started with Natural Language Processing", Manning (2022).
25. Rex B. Kline: "Principles and Practice of Structural Equation Modeling", Guilford Press (2023).
26. Ekaterina Kochmar: "Getting Started with Natural Language Processing", Manning (2022).
27. Jakub Langr, Vladimir Bok: "GANs in Action", Computer Vision Lead at Founders Factory (2019).
28. David Foster: "Generative Deep Learning", O'Reilly(2023).
29. Rowel Atienza: "Advanced Deep Learning with Keras: Applying GANs and other new deep learning algorithms to the real world", Packt Publishing (2018).
30. Josh Kalin: "Generative Adversarial Networks Cookbook", Packt Publishing (2018).
31. Thomas Haslwanter: „Hands-on Signal Analysis with Python: An Introduction“, Springer (2021).
32. Jose Unpingco: „Python for Signal Processing“, Springer (2023).
33. R. K. Burdick, C. M. Borror, D. C. Montgomery: "Design and Analysis of Gauge R&R Studies", 1st Ed. SIAM (2005); S. H. Derakhshan , C. V. Deutsch: "Numerical Integration of Bivariate Gaussian Distribution", Paper 405, CCG Anual Report 13 (2011).
34. C. Paar, J. Pelzl: "Understanding Cryptography", Springer (2010); H. Delfs, H. Knebl: "Introduction to Cryptography", 3rd Ed. Springer (2015); J. Katz, Y. lindell: "Introduction to Modern Cryptography", 2nd Ed, CRC Press (2015); O. Goldreich: "Foundations of Cryptography", Cambridge University Press (2008); J. P. Aumasson: "Serious Cryptography", no starch press (2018).
35. Kaggle-link: competition-documentation: <https://www.kaggle.com/competitions/drw-crypto-market-prediction>. R. Nystrom: „Game Programming Patterns“, 1st Ed. genever benning (2014); A. A. Stepanov, D. E. Rose: „From Mathematics to Generic Programming“, 1st Ed. Addison-Wesley (2015);
36. J. Berk, P. DeMarzo: „Corporate Finance“, 6th Ed., Pearson (2023); R. W. Melicher, E. A. Norton: "Introduction to Finance", 16th Ed. WILEY (2017); Anatoly B. Schmidt: "Quantitative Finance for Physicists: An Introduction", 1st Ed. Academic Press (2005); Alex Backwell: "An

Intuitive Introduction to Finance and Derivatives: Concepts, Terminology and Models", 1st Ed, Springer (2023); Michael Isichenko: "Quantitative Portfolio Management: The Art and Science of Statistical Arbitrage", 1st Ed., Springer (2021); John H. Cochrane: "Asset Pricing", Revised Ed., Princeton University Press (2005); Antti Ilmanen: "Expected Returns: An Investor's Guide to Harvesting Market Rewards", 1st Ed., WILEY (2011); Steven E. Shreve: "Stochastic Calculus for Finance I & II", 1st Ed., Springer (2004); Andrew Pole: "Statistical Arbitrage: Algorithmic Trading Insights and Techniques", 1st Ed., WILEY (2007); Mark S. Joshi: "The Concepts and Practice of Mathematical Finance", 2nd Ed., Cambridge University Press (2008);

37. E. Parzen: "Stochastic Processes", 3rd Ed. Dover Publications (2015); S. Aloorravi: "Metaprogramming with Python", 1st Ed. Packt (2022); B. Klein, P. Klein: "Funktionale Programmierung mit Python", Hanser (2025); K. Webel, D. Wied: "Stochastische Prozesse", 2. Auflage Springer (2016); L. Held: "Methoden der statistischen Inferenz", 1. Auflage Spektrum (2008); E. Cinlar: "Stochastic Processes", Dover (2013); N. Bäuerle, U. Rieder: "Finanzmathematik in diskreter Zeit", Springer-Spektrum (2017); M. Albrecht, R. Maurer: "Investment- und Risikomanagement", 3. Auflage, Schäffer Poeschel (2008); N. H. Bingham, R. Kiesel: "Risk Neutral Valuation: Pricing and Hedging of Financial Derivatives", 2. Auflage Springer (2004); T. Björk: "Arbitrage Theory in Continuous Time", 3rd Ed. Oxford University Press (2009); N. J. Cutland, A. Roux: "Derivative Pricing in Discrete Time", Springer (2013); F. Delbaen, W. Schachermayer: "The Mathematics of Arbitrage", Springer (2006); R. J. Elliott, P. E. Kopp: "Mathematics of Financial Markets", 2nd Ed. Springer (2005); H. Föllmer, A. Scheid: "A Stochastic Finance: An Introduction in Discrete Time", 3rd Ed. de Gruyter (2011); J. C. Hull: "Options, Futures and Other Derivatives", 8th Ed. Pearson (2011); J. Kremer: "Einführung in die diskrete Finanzmathematik", Springer (2005); D. Lamberton, B. Lapeyre: "Introduction to Stochastic Calculus Applied to Finance", Chapman & Hall (2007); D. G. Luenberger: "Investment Science", Oxford University Press (1998); S. R. Pliska: "Introduction to Mathematical Finance: Discrete Time Models", Blackwell (2000); A. N. Shiryaev: "Essentials of Stochastic Finance", World Scientific (2001); S. E. Shreve: "Stochastic Calculus for Finance I: The Binomial Asset Pricing Model", Springer (2004); J. Kremer: "Portfoliotheorie, Risikomanagement und die Bewertung von Derivaten", Springer (2011); L. Rüschendorf: "Mathematical Risk Analysis", Springer (2013).

38. A. Becker: "Kalman Filter - From the Ground Up", 1st Ed. private publication (2023); K. Triantafyllopoulos: "Bayesian Inference of State Space Models", 1st Ed. Springer (2021); P. Zarchan, H. Musoff: "Fundamentals of Kalman Filtering: A Practical Approach", 3rd Ed. AIAA (2009); A. Sidi: "Vector Extrapolation Methods with Applications", 1st Ed. SIAM (2019); C. Brezinski, M. R. Zaglia: "Extrapolation Methods - Theory and Practice", 2nd Ed. North-Holland (2002); C. Gardiner, P. Zoller: "Quantum Noise: A Handbook of Markovian and Non-Markovian Quantum Stochastic Methods with Applications to Quantum Optics", 3rd Ed. Springer (2004); K. Kendre: "Machine Learning for Quantum Noise Reduction", <https://arxiv.org/abs/2509.16242> (2025); D. C. Marinescu, G. M. Marinescu: "Classical and Quantum Information", 1st Ed. Academic Press (2012); Liao, H et al.: "Machine Learning for Practical Quantum Error Mitigation", arXiv:2309.17368v2 (2024), <https://arxiv.org/pdf/2309.17368>; Streamlit: <https://streamlit.io/>; Mitiq-package: <https://quantum-journal.org/papers/q-2022-08-11-774/>, <https://arxiv.org/abs/2009.04417>; Extrapolation packages: <https://pypi.org/project/extrapolation/>

39. A. Koop, H. Moock: "Lineare Optimierung - Eine anwendungsorientierte Einführung in Operations Research", 1st Ed. Spektrum (2008); G. B. Dantzig, M. N. Thapa: "**Linear Programming 1: Introduction**", 1st Ed. Springer (1997) & "Linear Programming 2: Theory and Extensions", 1st Ed. Springer (2003); H. S. Kasana, K. D. Kumar: "Introductory Operations Research, Theory and Applications", 1st Ed. Springer (2004); D. G. Luenberger: "Linear and Nonlinear Programming", 2nd Ed. Kluwer (2004); R. J. Boucherie, A. Braaksma, H. Tijms: "Operations Research - Introduction to Models and Methods", 1st Ed. World Scientific (2022); A. J. King, S. W. Wallace: "Modeling with Stochastic Programming", 2nd Ed. Springer (2024); J. O. Royset, R. J.-B. Wets: "An Optimization Primer", 1st Ed. Springer (2021); cvxpy package: <https://www.cvxpy.org/>, <https://pypi.org/project/cvxpy/>; py-packages for operations research: <https://wiki.python.org/moin/PythonForOperationsResearch>
40. (Py-)tesseract package: <https://github.com/tesseract-ocr/tesseract>, <https://pypi.org/project/pytesseract/>, <https://builtin.com/data-science/python-ocr>, <https://www.analyticsvidhya.com/blog/2024/04/ocr-libraries-in-python/> and UB Mannheim builds.
41. **Chip Huyen**, AI Engineering: Building Applications with Foundation Models, 1st Edition, O'Reilly Media, 2025; **Michael Lanham**, AI Agents in Action, 1st Edition, Manning Publications, 2025; **Melanie Mitchell**, Artificial Intelligence: A Guide for Thinking Humans, 1st Edition, Pelican Books, 2019; **Brian Christian & Tom Griffiths**, Algorithms to Live By: The Computer Science of Human Decisions, 1st Edition, Henry Holt and Company, 2016; **Ray Kurzweil**, The Singularity Is Nearer: When We Merge with AI, 1st Edition, Viking, 2024; OpenWeatherMap: <https://openweathermap.org/>, HuggingFace: <https://huggingface.co/>, LLM vs LRM: <https://www.aryaxai.com/article/llm-vs-lrm-vs-lam-understanding-the-future-of-language-based-ai-systems>, <https://magazine.sebastianraschka.com/p/understanding-reasoning-llms>